

ADD City Lab | 成都站

AI+研发数字峰会

AI+ Development Digital Summit

AI引领研发，智创无限可能

🕒 2026年9月19日 📍 成都首座万豪酒店



在不确定性中重建软件工程确定性

邱娟 | Inspire (前 Thoughtworks 中国)

邱娟

AI4SE首席顾问 / 博士 / CCF 高级会员



研究与学术

- 深耕软件工程与研发效能逾十八年
- 发表 SCI/EI 论文；参与头部企业科研

产业实践

- 金融科技超大规模代码重构与产品化
- Hackathon 获奖；创业公司 CTO 经历

专业身份

- CCF 高级会员；信通院 MLOps 指导专家
- 华师大企业讲师；SECON+ / TiD 受邀分享

当前工作

- 企业 AI4SE：基线 → 试点 → 赋能
- 把 AI Coding / Agent 做成可度量能力

目录 CONTENTS

- I. 看见不确定性
- II. 确定型工程范式 · 四层模型
- III. 确定性实践方向 · 四个控制面
- IV. 转型落地路径 · 如何落地与证明



PART 01 AI4SE带来的不确定性



软件工程的范式正在经历前所未有的变革，范式在变，术语在涨，工具在卷：

Prompt Engineering, Context Engineering, Harness Engineering, Loop Engineering, Graph Engineering, Spec Driven Development, Vibe Coding, Agentic Engineering, Ralph Loop, Human-in-the-loop (HITL)/HOTL, AI-Native SDLC, Autonomous Coding, Software 1.0/2.0/3.0 ...

热词网红：Andrej Karpathy、Mitchell Hashimoto、Geoffrey Huntley、Addy Osmani、Simon Willison ...

工具琳琅满目：GitHub Copilot、Cursor、Claude Code、Codex、Devin、Windsurf、Google Antigravity、Gemini Code Assist、Amazon Q Developer、Manus、Jules、Cline、Continue、Aider、OpenHands ... 等数十种 Coding Agent

研发同学身处技术的洪流中，既兴奋又迷茫，努力追逐却又时常感到掉队？组织如何选型、如何落地、如何规模化？

“穿透表象，把握本质，在变革中锚定方向”



► 软件工程正在进入新一轮技术颠覆

AI 正在改变软件工程的接口、流程与组织方式



AI 类似历史上的重大技术周期，但扩散更快、实践更不稳定，更适合主动试点而不是被动观望。

参考 The Pragmatic Engineer: Martin Fowler & Kent Beck: Frameworks for reinventing software, again and again

(ref: <https://www.youtube.com/watch?v=CZs8J1ZD0CE>)

<https://newsletter.pragmaticengineer.com/p/cycles-of-disruption-in-the-tech>

▶ 没有人拥有唯一标准答案

工具、模型、流程和组织模式都还在高速演化

工具先驱

Claude Code、Codex、Cursor 等工具证明了 agentic coding 的方向。

方法探索

SDD、Agentic Engineering、Harness Engineering 都在提供重要视角。

企业差异

大型组织不能直接复制个人开发者或创业团队的使用方式。

工作建议

需要在真实环境中适配业务、代码库、流程、文化与治理边界。

结论：AI4SE 的最佳路径需要由企业自身在真实工程环境中验证出来。

咨询不是万能模板，却在不确定性中帮助企业完成适配

01

识别不确定性

趋势、工具、流程和组
织约束
共同构成试点假设

02

设计适配路径

结合企业现状选择场景、
团队、指标和治理边界

03

陪跑真实试点

在真实代码库与真实交
付任务中验证 workflow

04

沉淀自有方法

形成可复制资产、度量
看板和推广路线图

降低试错成本

避免盲目采购和无序试用，提前识别质量、安全、
成本和采纳风险。

沉淀组织能力

将个人经验转化为可复制 workflow、rules、skills
与持续改进机制。

PART 02 AI4SE 的确定型工程范式



每一层都依赖下一层；越往上越显眼，越往下越关键。



AI4SE 升级判断

01 地基升级

Quality Focus →
Effectiveness
Quality + Efficiency

02 横切协作

Harmony 贯穿 Process /
Methods / Tools

03 执行者升级

从「人」扩展为「人 + Agent」

04 方法论归位

Harness、SDD、Agentic
Engineering → Methods

结构继承；内涵从「工具采购」升级为「交付系统升级」。



Effectiveness Focus

工程系统首先追求什么结果？



01

随后：地基升级为 **Effectiveness = Quality + Efficiency**

► 地基：Effectiveness = Quality + Efficiency

Quality 含 Trust; Efficiency 含 Value。Efficiency 不是「快」，而是无浪费地产生有效成果。

Quality（做得对、做得稳）

正确性、可维护、可验证
可审计、可回滚

Trust：权限、安全、合规

AI 时代需要独立验证证据

Efficiency（无浪费地产生成果）

流动效率、小批量、吞吐
减少返工与等待浪费

Value：交付对业务有效的成果

快但做错 ≠ Efficiency

组织地基 = 愿意为速度牺牲多少质量？愿意为保证质量投入多少效率成本？



Process

活动如何形成可审计闭环?



02

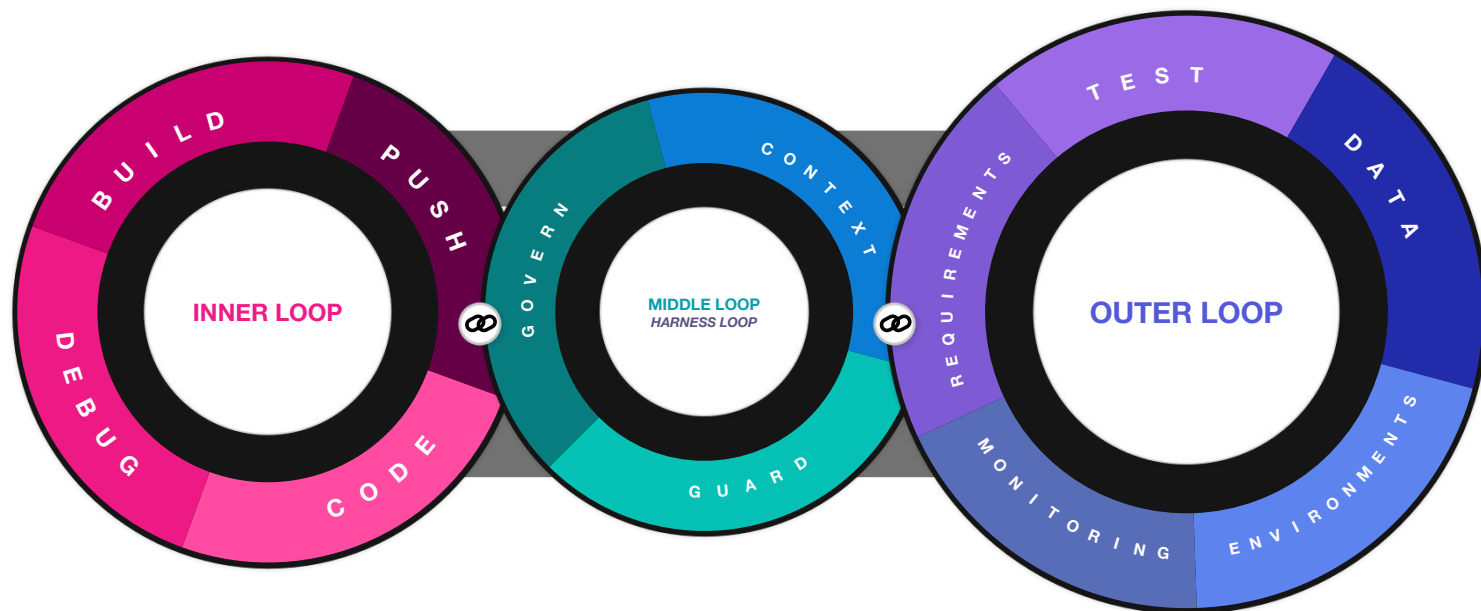
随后: **Process** 升级与三环协同模型

► Process: 从活动序列到可审计闭环

证据驱动的人机协作流程；各阶段路由见后文 HITL / HOTL / HOOL。



► 实施指导思想：三环协同模型



核心逻辑：以三环协同为整体框架，不预设单一核心；通过试点探索三者配合模式，再为规模化推广提供实践依据。

Inner Loop（开发本地闭环）

CODE → BUILD → DEBUG → PUSH

核心是开发者与AI协同的极速验证链路，专注代码实现与本地验证。

Middle Loop（监督治理闭环）

Context → Guard → Govern

核心是工程师作为“监督者”，为AI提供上下文、校验输出、控制风险。

Outer Loop（持续交付与运维运营闭环）

REQ → TEST → DATA → ENV → MONITORING

CI/CD、部署与运维监控与运营。AI打通流程、沉淀经验，形成可推广的工程实践。



Methods

方法论如何归位到过程层?



03

随后: **Methods** 归位——方法论族如何安放

Methodology 规定纪律；Tools 提供机制。Harness 归入 Methods，不在 Tools 单独成层。

Agentic Engineering
Agent 成为工程参与者

Commands · Skills · Subagents

Harness Engineering
驾驭模型、约束 Agent

AGENTS.md · Hooks · Permissions

SDD
规格驱动变更与验收

OpenSpec · proposal/tasks

Loop Engineering
可治理交付闭环

Inner / Middle / Outer Loop 自治

传统方法
人的工程纪律

TDD · 重构 · 代码评审

Tools

工具层如何兼容并包、支持集成?



04

随后: Tools workflow 地图, 再进入 Harmony

精选生态地图：按类型与工作流理解工具位，服务于 Process 与 Methods。

Coding Agent 类型

IDE

Cursor
GitHub Copilot
Windsurf

CLI

Claude Code
Aider
Codex CLI

Cloud

Devin
Codex Cloud
Replit Agent

Open Source

OpenHands
Cline
Continue

工作流模式

Plan-first

先规划再实现，降低盲目生成

Spec-first

先规格再编码，对齐验收边界

Test-first

先测试再实现，用证据约束输出

Multi-agent

多 Agent 分工协作，人保留裁决

参考：<https://artificialanalysis.ai/agents/coding> · <https://github.com/shanraissshan/claude-code-best-practice>

► Harmony: 把人机协作写成制度

Harmony 不是第五层，而是贯穿 Process / Methods / Tools 的横切契约。

角色	边界	接力	问责	审批
Planner · 规划	自治动作	Spec · Plan	合并 · 发布	权限 · Hooks
Generator · 生成	人工介入	Diff · Test Log	安全责任	沙箱 · 白名单
Evaluator · 验收	风险升级	Review Record	人类始终担责	审计留痕

把协作原则写成可执行、可验证、可审计的制度。

按风险路由： HITL / HOTL / HOOL

按动作风险选择 HITL / HOTL / HOOL，而非全程使用一种模式。

阶段	主角色	路由	门禁
Research	人 + Agent	HOTL/HOOL	探索草稿；人监督方向与边界
Plan	人主导	HITL	Spec / Plan 签署后执行
Execute	Agent 主导	HOTL	沙箱小步实现；风险升级 HITL
Review	独立 Evaluator	HITL	人不可省略；Agent 不可自评
Ship	人担责	HITL	合并 / 发布 / 生产变更由人批准

哪一步必须由人批准？
哪些动作可以自治，风险何时升级？
证据、责任与审计记录是否完整？

► 范式总览：如何理解 AI4SE 工程系统

上一章回答如何理解工程系统。与 Pressman 对照：结构继承；地基升级为 Effectiveness；Harmony 以侧向方框横切三层。



相对 Pressman 的三处演进

地基：Quality Focus → Effectiveness (Quality + Efficiency)

横切：新增 Harmony (Human ↔ Agent) — 侧方框贯穿

Process / Methods / Tools

内涵：三层执行者从「人」扩展为「人 + Agent」

Methods 层容纳 Harness、SDD、Agentic Engineering 等方法论族

Tools 层承载 Agent 运行时与 Harness 机制

Harmony 不是第五层叠盘，而是协作契约。

PART 03 AI4SE 的确定性实践方向



► 四个控制面：企业具体建设什么

SDD 管输入，Agentic 管执行，Harness 管环境治理，Operating Model 管规模化



四个控制面不是并列概念堆叠，而是一条从输入、执行、治理到组织化的能力链。

本章回答企业具体建设什么。

► SDD: 先管住输入质量

把需求从文档升级为 agent 可执行输入资产

Before

传统需求输入

自然语言含混，依赖个人理解
验收标准和非功能要求经常滞后
AI 直接生成代码，返工风险高



After

AI 可执行规格

明确目标、约束、接口和验收条件
产品、架构、开发、测试、安全先对齐
输出更容易评审、测试和追踪

Spec

Plan

Tasks

Implement

Verify

► Agentic Engineering: 进入真实任务流

关键是把 agent 放入真实研发流程，而不是只做演示

代码库探索

理解上下文

方案生成

生成候选方案

测试补齐

补齐边界用例

构建修复

定位失败原因

评审辅助

发现质量风险

文档同步

同步知识资产

从 “AI 帮我写几行代码” 转向 “AI 参与研发任务闭环”。

人类负责判断与责任，agent 负责检索、候选实现和验证辅助

人类工程师

- 定义目标和业务边界
- 判断上下文和架构取舍
- 审查结果并承担质量责任



AI Agent

- 检索代码与文档上下文
- 生成候选方案和初步实现
- 辅助测试、修复和知识整理

► Harness: 让执行可监督、可验证、可治理

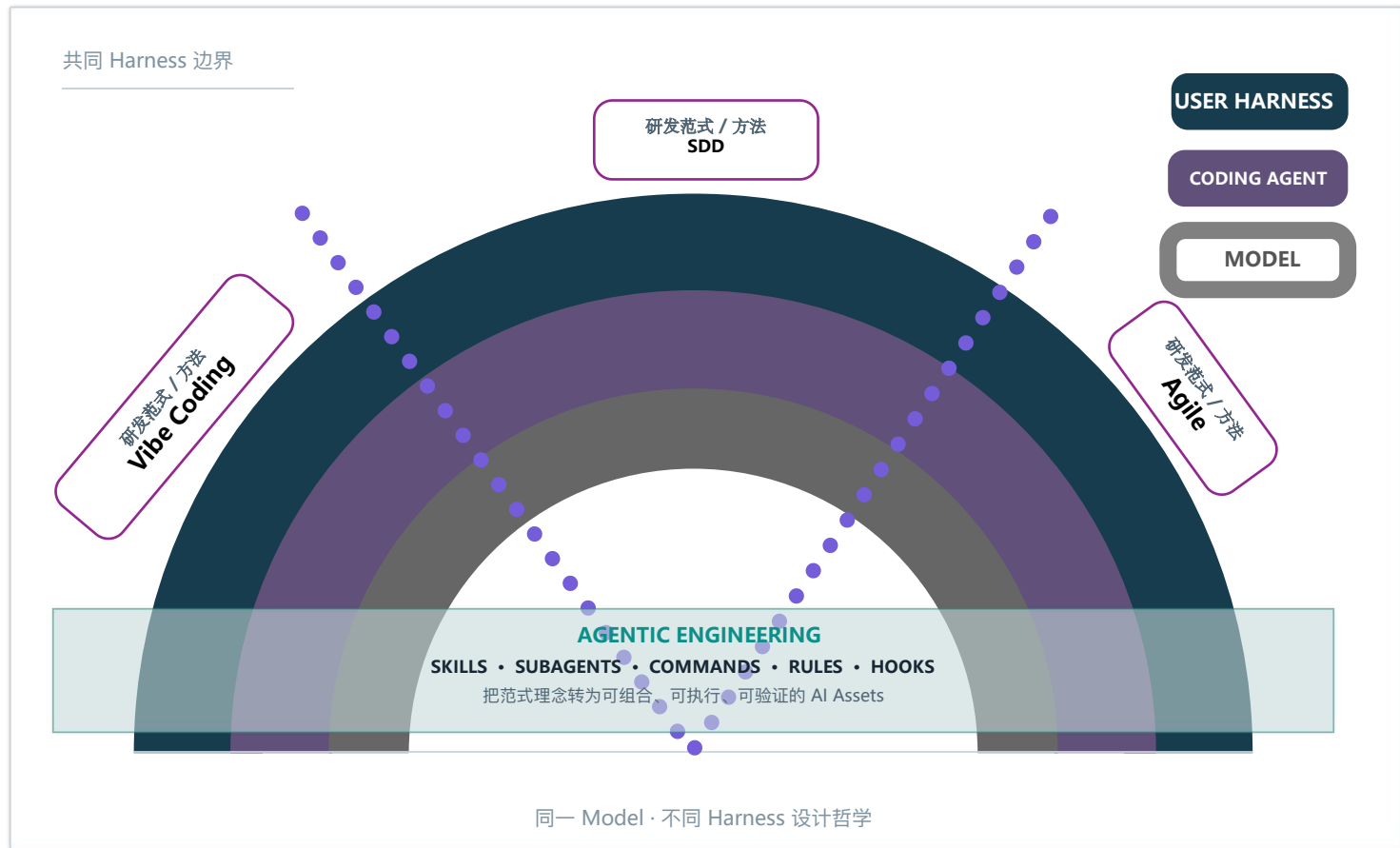
围绕 agent 构建上下文、工具、权限、反馈和治理环境





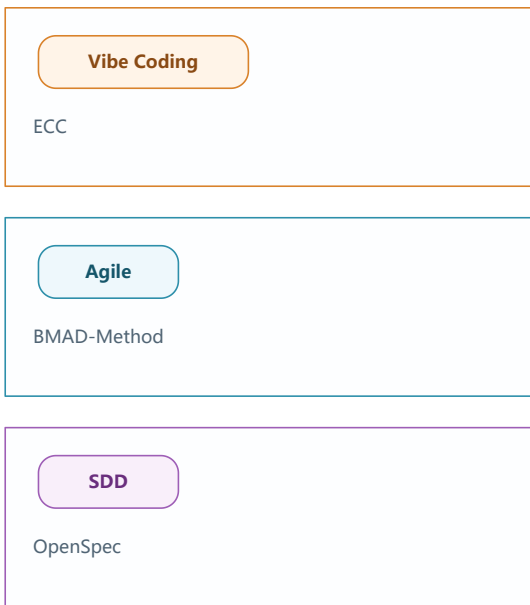
开发范式如何决定 Harness 策略

开发范式定调 AI4SE Harness 策略, Agentic Engineering 践行 IT4IT 理念



学习参照 (示例)

用于学习与比较, 不代表唯一实现



Agentic Engineering 不是第四种范式,
而是把方向落成 AI Assets 的工程方式。

Model 相对稳定; 外两层随范式按需组合。

$$\mathcal{H}^p(x) = \text{UserHarness}^p(\text{CodingAgent}^p(\text{Model}(x)))$$

参考: <https://martinfowler.com/articles/harness-engineering.html>



PART 04 AI4SE 的转型落地路径

► 组织如何真正转向 AI4SE

回答组织如何转型，以及如何证明转型有效。

01 各角色如何转型。

02 敏捷团队运作方式如何转型。

03 如何从试点扩展为组织级能力。

04 如何科学度量相对传统研发方式的提效。

从传统角色迁移到四类 AI4SE 职能；四类角色是职能，不一定对应四个人；执行者与验证者不能完全合并。

传统角色	新职能	价值重心变化	不可越过的责任边界
Product Owner	Intent Owner (IO) 需求意图与价值负责人	从管理 Backlog 转向定义意图与成功标准	成功标准与意图裁决权不可外包给 Agent
架构师/分析师	Spec Engineer (SE) 规格工程师	维护 Living Spec、一致性和规格质量	规格一致性与验收边界不可省略人工裁决
研发效能/AI 平台	AI4SE Engineer AI4SE效能工程师	建设 Harness、SKILL、Recipe 和 Agent 资产	资产升格与治理规则需人确认后生效
QA/Test Lead	IVE 实施验证工程师	设计独立验证体系并持有质量关卡裁决权	执行者与验证者不能完全合并

► 运作方式如何转型：从 Sprint 到 Change Flow

核心不是抛弃敏捷，而是从 Sprint 时间盒驱动转向 Change Flow 变更驱动。

Before

- Sprint 时间盒
- Backlog 驱动
- 编码为瓶颈
- 末端测试

After

- **Change Flow**
- **Living Spec**
- 定义/验证为瓶颈
- **G1 / G2 / G3**

过渡态：两周 **Scrum** 过渡态 —— 保留透明、检视、适应，逐步把时间盒驱动迁移为 **Change Flow**。

► 团队 Golden Path：可复制的变更流

用 5+1 阶段组织团队级变更流；每阶段只显示主导角色、关键工件和退出关卡。



退出关卡：G1 规格对齐 · G2 验证完成 · G3 资产沉淀

复制对象不是 AI 工具，而是经过验证的角色、流程、规则、能力资产和度量口径。

01 试点探索

进入
选定真实小变更与基线口径

产出
可运行 Change Flow
角色分工与初版度量

退出
完成至少一轮可验证闭环

02 沉淀巩固

进入
试点指标改善且治理可审计

产出
固化 Rules
SKILL
Recipe 与退出条件

退出
资产可复用且边界清晰

03 规模化推广

进入
复制对象已验证
(角色/流程/规则/资产/口径)

产出
跨团队推广与健康度治理

退出
指标不改善或治理失守则停止扩面

RULE → SKILL → AGENT → COMMAND → HOOK; 不成熟不升格。



原则：不成熟不升格。

RULE 先立规矩，再沉淀 **SKILL / AGENT**，最后才升格为 **COMMAND** 与 **HOOK**。

复用率

命中率

漂移率

废弃率

► 用 Throughput 与 Cycle Time 证明提效

用双指标衡量提效，并设置质量护栏避免只追求速度。

Throughput

单位时间内完成验证的 Change / Verified Increment 数

越高越好

Cycle Time

从 Spec 定义完成到形成 Verified Increment 的时间

越短越好

质量护栏（避免只追求速度）

一次验证通过率

缺陷逃逸率

返工率

匿名试点：方向性证据，而非普适承诺

匿名试点评估：归一化指数；仅代表该场景，不外推行业 ROI。



Throughput 1.0 → ≈2.0 (提升约 100%) ; Cycle Time 1.0 → ≈0.5 (周期缩短约 50%) 。

一场快速收束：看见不确定性 → 理解AI4SE范式 → 再抓控制面 → 有节奏地规模化。

01 没有银弹

没有银弹可买；只能在不确定性中重建确定性——靠工程系统能力，而不是单点工具。

02 理解AI4SE范式

Effectiveness = Quality + Efficiency; 再以 Process / Methods / Tools + Harmony 理解 AI4SE 确定型工程范式。

03 再抓控制面

抓住四个控制面：SDD 管输入、Agentic 进任务流、Harness 可监督验证、Operating Model 可运作。

04 有节奏地规模化

先小变更试点，设基线与退出条件；用 Throughput / Cycle Time 证明后，再有节奏地扩面。

NiDD City Lab | 成都站

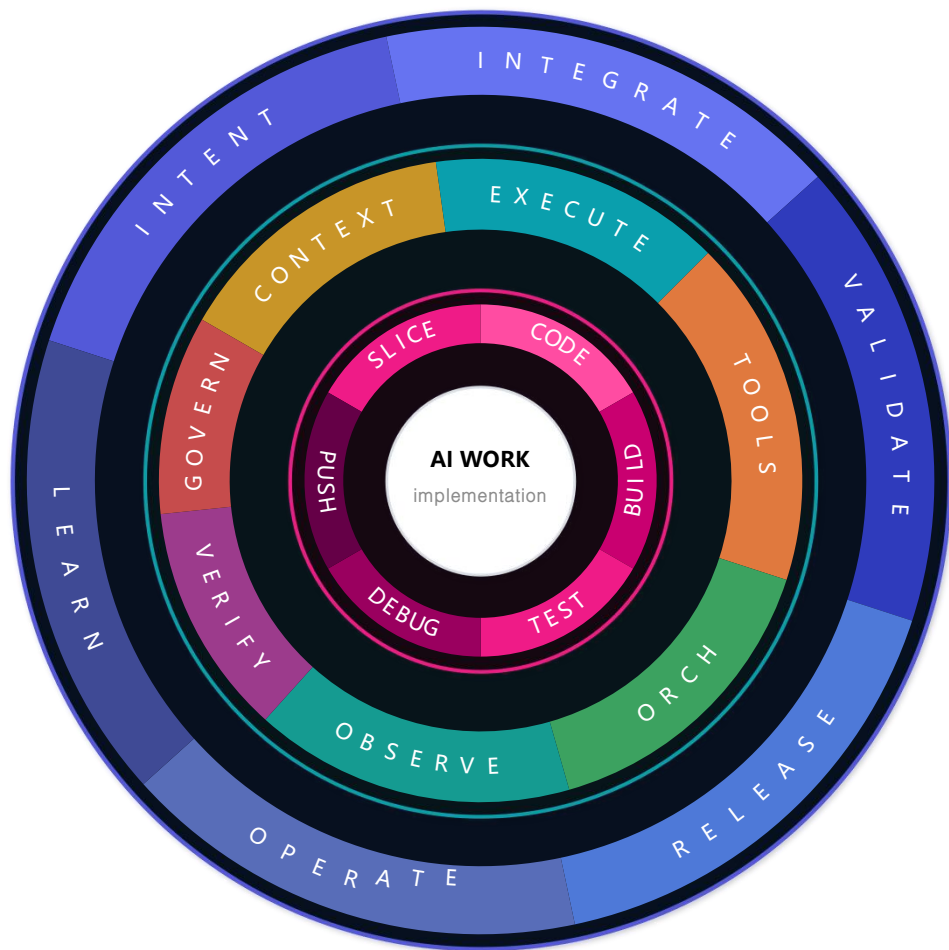
AI+研发数字峰会
AI+ Development Digital Summit

扫码领取会议PPT资料

扫码领取会议PPT资料



► The Three Loops



OUTER LOOP

从候选变更进入组织级交付系统，
到生产运行、反馈和治理改进。

Defines business intent, risk boundaries,
release policy, and production feedback.

MIDDLE / HARNESS LOOP

把AI/Agent的工作变成可监督、
可验证、可治理的工程产出。

Controls agent execution: context, tools,
orchestration, observability, verification, governance.

INNER LOOP

从明确小任务出发，快速产生可运行、
可测试、可提交的候选变更。

Produces candidate changes through coding,
build, test, and debug feedback.

参考: <https://picrew.github.io/LLM-Harness/> · <https://annievella.com/posts/the-middle-loop/>